

Harvests Bilingue

Faire passer toute la plateforme — vitrine publique, tableaux de bord métiers, back-office, e-mails et chatbot — du français seul au français / anglais, sans casser ce qui existe déjà.

- **Portée** Frontend React (Vite) · API Node/Express · e-mails Pug · chatbot node-nlp
- **Langues cibles** fr (défaut actuel) → fr + en • **Base** audit du dépôt au 08/09/2026
- **Durée estimée** 24–66 j selon staffing (détail §03)

◆ 01 — CONSTAT

L'infrastructure existe déjà, l'adoption non

i18next côté frontend et le module i18n côté backend sont installés et partiellement configurés depuis un chantier précédent. Le contenu réel, lui, est presque entièrement resté câblé en dur en français — c'est le vrai chantier.

frontend 9 / 474 fichiers .jsx/.js utilisant déjà useTranslation	backend 2 / 57 contrôleurs passés par le helper i18nResponse	0 / 8 gabarits d'e-mail (.pug) traduits	1 langue chatbot node-nlp entraîné en "fr" uniquement
156 / 68 pages métiers vs. pages back-office admin	14 pages publiques déjà câblées à SEOHead / useSEO		

◆ 02 — LE CHANTIER

Dix étapes, regroupées comme un cycle de culture

Même logique que les fiches de culture de la plateforme : on prépare le terrain, on construit, on étend la couverture, puis on récolte et on consolide.



SEMIS poser les fondations avant d'écrire le moindre texte anglais

0

Cadrage & décisions de fondation

Bloquant

1-2 j

Trancher les questions qui, si elles bougent plus tard, obligent à tout refaire — avant de migrer un seul composant.

- ☐ Langue par défaut = `fr`, anglais en bascule explicite (pas de géo-détection forcée au premier chargement)
- ☐ Stockage de la préférence : `localStorage` (déjà géré par `utils/i18n.js`) + cookie `harvests_lang` partagé avec le backend (déjà scaffoldé dans `backend/config/i18n.js`) + champ `preferredLanguage` sur le modèle `User` pour la persistance long terme et les e-mails
- ☐ Pas de préfixe d'URL par langue (`/en/...`) dans un premier temps — trop de refonte de routing pour la valeur ajoutée à ce stade
- ☐ Choisir le pipeline de traduction : brouillon assisté puis relecture humaine native obligatoire avant mise en prod (pas de publication de traduction automatique brute)

Bloque toutes les phases suivantes

1

Ossature technique

Bloquant

3-5 j

Rendre l'infrastructure existante prête à absorber des centaines de clés sans devenir un fichier plat ingérable.

- ☐ Éclater `frontend/src/locales/{fr,en}.json` (~780 lignes chacun) en namespaces par domaine : `common`, `auth`, `dashboard.producer`, `dashboard.admin`, `products`, `orders`, `chat`, `errors`
- ☐ Ajouter un sélecteur de langue dans `DashboardTopbar` et dans le header public, relié à `i18n.changeLanguage()`
- ☐ Mettre en place un script CI qui détecte les clés manquantes entre `fr.json` et `en.json` (build cassé si désynchronisé)
- ☐ Ajouter une règle de lint (regex ou `eslint-plugin-i18next`) qui repère les chaînes françaises codées en dur dans le JSX, pour empêcher la dette de revenir après coup

Dépend de Phase 0



2

Pages publiques & référencement

Élevée

5-8 j

Prioritaire pour l'acquisition : ce sont les 14 pages déjà indexées par les moteurs de recherche.

- ☐ Traduire les clés `seo.*` consommées par `useSEO.js` pour Home, Products, Categories, Producers, Pricing, Blog, Contact, About, etc.
- ☐ Ajouter les balises `hreflang` alternées (fr/en) via `SEOHead.jsx`
- ☐ Générer un sitemap par langue
- ☐ Traduire le contenu réel des ~185 composants qui portent ces pages (le texte vit dans `components/home/*`, pas dans les fichiers `pages/*` eux-mêmes)

Dépend de Phase 1

3

Tableaux de bord métiers

Élevée

15-25 j

Le gros du volume : producteur, transformateur, transporteur, restaurateur, consommateur, exportateur.

- ☐ Découper en sous-lots par rôle, dans l'ordre d'usage réel : onboarding & authentification d'abord, puis produits/commandes/panier, puis pages secondaires (avis, stats, paramètres)
- ☐ Commencer par les gabarits partagés (`ModularDashboardLayout`, `DashboardSidebarFixed`, `DashboardTopbar`) pour que la bascule de langue soit visible immédiatement partout
- ☐ Réutiliser les fiches de culture déjà bilingues comme modèle : `cropAdviceData.js` est un bon candidat pour prouver le pattern avant de l'étendre
- ☐ Revue systématique des formulaires (validation, placeholders, messages d'erreur inline) — souvent oubliés dans une première passe de traduction

Dépend de Phase 1 · parallélisable avec Phase 2

4

Réponses API & validations

Élevée

6-9 j

Le backend a déjà le détecteur de langue et les helpers — il manque juste l'adoption dans les contrôleurs.

- ☐ Étendre `res.success` / `res.error` (`backend/middleware/i18nResponse.js`) aux 57 contrôleurs identifiés avec du français en dur, module par module (auth en premier — `authController.js`, `passwordController.js`, `authMiddleware.js`)
- ☐ Basculer les messages de validation (Joi/Mongoose) sur les clés de `backend/locales/{fr,en}.json`
- ☐ S'assurer que le middleware `detectLanguage` lit le cookie `harvests_lang` défini en Phase 0, pas seulement l'en-tête `Accept-Language`

Dépend de Phase 0



5

E-mails transactionnels

Moyenne

2-3 j

8 gabarits Pug, tous en français en dur, sans aucun appel de traduction.

- ☐ Injecter `user.preferredLanguage` dans le contexte de rendu de chaque gabarit (`welcome` , `passwordReset` , `accountApproval` , `orderConfirmation` , `incompleteProfile` , `subscriptionExpired` / `Expiring` , `mailing`)
- ☐ Dupliquer le texte en versions fr/en via les clés backend plutôt que forker les fichiers .pug (un seul gabarit, deux jeux de chaînes)
- ☐ Mettre à jour l'attribut `html(lang=...)` dynamiquement

Dépend de Phase 4

6

Chatbot bilingue

Moyenne

3-5 j

Le `NlpManager` de `chatBotController.js` n'est configuré qu'avec `languages: ["fr"]` — l'anglais n'existe pas dans le modèle.

- ☐ Ajouter `"en"` à la configuration du manager, dupliquer chaque `addDocument` / `addAnswer` (intentions FAQ, salutations, remerciements, au revoir) en anglais
- ☐ Router `manager.process()` sur la langue de la conversation plutôt que sur `"fr"` codé en dur
- ☐ Ré-entraîner et régénérer `model.nlp` , valider qu'il reste synchronisé entre la racine du repo et `backend/model.nlp`

Dépend de Phase 0

7

PWA, manifeste & périmètre admin

Différée

1-2 j

Deux décisions de périmètre plus qu'un chantier de traduction pur.

- ☐ Traduire `frontend/public/manifest.json` (name, description, shortcuts) — nécessite soit un manifeste généré par langue, soit une version bilingue neutre
- ☐ Décider si le back-office (68 fichiers sous `pages/admin` + `components/admin`) est traduit ou reste volontairement français-only, l'équipe interne étant francophone — recommandé : différer et ne traduire que si un membre d'équipe non-francophone rejoint

Dépend de Phase 1



8

QA, glossaire & gouvernance

Élevée

3-4 j

Un glossaire commun évite que deux traducteurs inventent chacun leur terme pour la même notion métier.

- ☐ Figurer un lexique de référence (voir tableau ci-dessous) partagé entre l'équipe produit, les traducteurs et le contenu des e-mails/chatbot
- ☐ Relecture native anglophone obligatoire avant publication de toute clé **en**
- ☐ Test de bascule de langue sur les parcours critiques : inscription, paiement, suivi de commande, chat

Dépend de Phases 2 à 6

9

Lancement progressif

Moyenne

2-3 j

Sortir la bascule de langue au fur et à mesure que chaque module atteint une couverture suffisante, plutôt qu'en un seul big-bang.

- ☐ Activer le sélecteur de langue d'abord sur les pages publiques (Phase 2), le reste du site restant en français tant que non couvert
- ☐ Étendre au fil de l'eau : dashboards, puis e-mails, puis chatbot
- ☐ Suivre le taux d'usage de la bascule anglais pour prioriser les zones encore non traduites signalées par les utilisateurs

Dépend de Phase 8

PRIORITÉ

Bloquant

gate les autres phases

Élevée

impact utilisateur direct

Moyenne

important, non urgent

Différée

à trancher plus tard

DURÉE en jours ouvrés de développement, hors délai de relecture externe

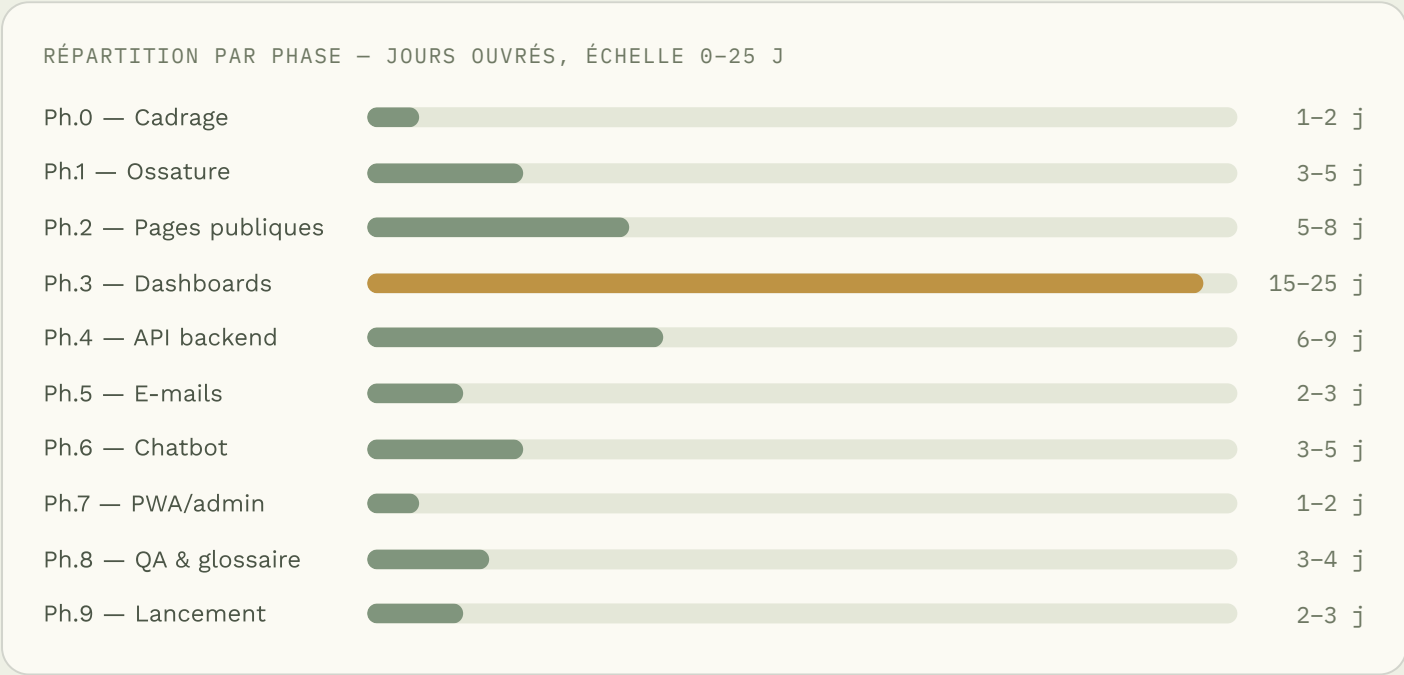
◆ 03 – TEMPS NÉCESSAIRE

41 à 66 jours seul·e, 24 à 39 en équipe

Estimation en jours ouvrés de développement pur (hors turnaround de relecture native, qui dépend de la disponibilité du traducteur). Les phases 2, 3 et 4 sont

indépendantes une fois la Phase 1 posée : c'est là que la parallélisation rapporte le plus.

41–66 j 1 développeur·se, tout le chantier en séquentiel — soit ≈ 8–13 semaines	24–39 j équipe de 2–3, phases 2/3/4 et 5/6/7 menées en parallèle — soit ≈ 5–8 semaines	15–25 j chemin critique en équipe : les tableaux de bord métiers (Phase 3) dominant, tout le reste tient dans leur ombre
---	--	--



◆ 04 — LEXIQUE DE RÉFÉRENCE

Un même mot, une seule traduction

Extrait du vocabulaire métier de la plateforme à figer avant de traduire en masse
— évite les incohérences entre l'interface, les e-mails et le chatbot.

FRANÇAIS	ENGLISH	CONTEXTE
Producteur	Producer	Rôle utilisateur, dashboards, filtres catalogue
Transformateur	Processor	Rôle utilisateur — éviter "Transformer" (ambigu, homonyme technique)
Transporteur	Carrier	Rôle utilisateur — "Transporter" fonctionne aussi, "Carrier" plus naturel en anglais logistique
Filière	Value chain	Pages "Investisseurs", contenu institutionnel
Fiche de culture	Crop guide	Module conseils agricoles (cropAdviceData.js)
Étape de pousse	Growth stage	Les 4 étapes illustrées par culture (semis / croissance / floraison / récolte)
Zone Sahélienne	Sahel zone	Bandeau contextuel des fiches de culture
Récolte	Harvest	Nom de la plateforme — ne jamais traduire "Harvests" lui-même

◆ 05 – POINTS DE VIGILANCE

Ce qui fait dérailler ce genre de chantier

Dette qui revient en douce

Sans le lint CI de la Phase 1, chaque nouvelle fonctionnalité réintroduit du français en dur et l'écart avec l'anglais grandit plus vite qu'on ne le comble.

Traduction sans relecture native

Une traduction techniquement correcte mais qui sonne faux dégrade la confiance plus qu'une page restée en français. La relecture (Phase 8) n'est pas optionnelle.

fr.json / en.json qui divergent

Un fichier plat de 780 lignes par langue, sans vérification automatique des clés manquantes, part en désynchronisation silencieuse dès le deuxième contributeur.

Chatbot à moitié bilingue

Répondre en anglais avec un modèle entraîné uniquement en français produit des réponses hors-sujet plutôt qu'une absence de réponse — pire pour l'utilisateur que pas de bascule du tout.